Rapport de Projet

Année universitaire — Revue numérique associative

Un éditeur de documents partagé

Conception et réalisation d'une application web de rédaction collaborative et de distribution documentaire contrôlée

Problématique centrale

Concevoir un système de rédaction collaborative et de distribution documentaire garantissant la cohérence des données, le respect des règles de gestion et la maîtrise des accès.

Association — Revue numérique annuelle 25 octobre 2026

Résumé

Ce rapport présente la conception d'une application web destinée à une association qui publie chaque année une revue numérique rédigée de manière collaborative par plusieurs centaines de rédacteurs. L'application permet la création, le partage et l'édition simultanée de documents textuels, la gestion des versions, ainsi que la distribution des documents au format PDF au moyen de liens de téléchargement dont le quota d'utilisation est strictement contrôlé.

Le rapport identifie les acteurs, précise le périmètre fonctionnel, énonce les exigences fonctionnelles et non fonctionnelles, formalise les principes métier, puis présente l'analyse et la conception UML (cas d'utilisation, classes, activités, états-transitions). Deux contraintes sont identifiées comme critiques et non négociables : le respect strict du quota des liens de téléchargement, et la conservation de toute modification acceptée en édition concurrente.

Mots-clés : édition collaborative, distribution documentaire, PDF, quotas, concurrence, cohérence des données, UML, AWS.

Table des matières

Résumé i

Table des matières iii

Table des figures iv

Liste des tableaux v

1 Présentation générale du projet 1
- 1.1 Contexte . . . 1
- 1.2 Problématique . . . 1
- 1.3 Objectif général . . . 1
- 1.4 Objectifs spécifiques . . . 1

2 Identification des acteurs 3
- 2.1 Le rédacteur . . . 3
- 2.2 Le visiteur téléchargeur . . . 3
- 2.3 L'administrateur de l'association . . . 3

3 Périmètre fonctionnel 4
- 3.1 Fonctionnalités incluses . . . 4
- 3.2 Limites du périmètre . . . 4

4 Exigences fonctionnelles 5

5 Exigences non fonctionnelles et critères mesurables 6
- 5.1 Contraintes critiques et majeures du projet . . . 6
- 5.2 Contraintes d'infrastructure et règles d'évaluation . . . 7
- 5.3 Disponibilité et capacité : objectifs chiffrés mesurables . . . 8
- 5.4 Sécurité et gestion des accès . . . 8
  - 5.4.1 Authentification de l'administrateur . . . 8

6 Principes métier fondamentaux 9
- 6.1 Principe de cohérence documentaire . . . 9
- 6.2 Principe de validité du lien . . . 9
- 6.3 Principe de respect strict du quota . . . 9
- 6.4 Principe de conservation des modifications . . . 10
- 6.5 Principe de limitation de l'écriture . . . 10
  - 6.5.1 Précisions sur le crédit d'écriture . . . 10
- 6.6 Principe de blocage en cas de tentative de contournement . . . 10
- 6.7 Principe de gestion des versions et de l'historique . . . 11
- 6.8 Principe de droits d'accès . . . 11
- 6.9 Principe de la notification au créateur . . . 11
- 6.10 Principe des paramètres du lien . . . 12

7 Analyse et conception **13**
7.1 Diagramme de cas d'utilisation . . . 13
7.1.1 Descriptions textuelles des cas d'utilisation . . . 13
7.2 Diagramme de classes . . . 22
7.3 Diagrammes d'activités . . . 22
7.3.1 Distribution d'un document via un lien . . . 22
7.3.2 Modification d'un document . . . 23
7.4 Diagrammes d'états-transitions . . . 23
7.4.1 Cycle de vie d'une demande de téléchargement . . . 23
7.4.2 Cycle de vie d'un document et des versions . . . 24

8 Architecture Cloud AWS **25**
8.1 Présentation générale de l'architecture retenue . . . 25
8.2 Justification et rôle de chaque service AWS . . . 25
8.2.1 Amazon EC2 : hébergement de l'application et de la collaboration temps réel . . . 25
8.2.2 Amazon DynamoDB : persistance d'état et décompte atomique . . . 26
8.2.3 Amazon SQS : découplage asynchrone de la compilation documentaire . . . 26
8.2.4 Amazon S3 : stockage objet hautement disponible et URLs présignées . . . 27
8.2.5 Amazon SNS : notification événementielle découplée . . . 27
8.2.6 AWS IAM : sécurité par le moindre privilège et IMDSv2 . . . 27
8.2.7 Amazon CloudWatch : observabilité et métriques d'évaluation . . . 28
8.3 Synthèse des avantages majeurs de l'architecture . . . 28

9 Architecture Logicielle et Choix Technologiques **29**
9.1 Vue d'ensemble du découpage applicatif . . . 29
9.2 Justification détaillée des choix technologiques Frontend . . . 29
9.2.1 Framework d'interface : React 18 couplé à Vite . . . 29
9.2.2 Moteur d'édition de texte riche : Quill.js (via y-quill) . . . 29
9.2.3 Moteur de synchronisation collaborative : Yjs (CRDT) . . . 30
9.2.4 Gestion de la présence en temps réel : Sous-protocole Awareness de Yjs . . . 30
9.2.5 Contrôleur de téléchargement visiteur : Machine à états finis . . . 30
9.3 Justification détaillée des choix technologiques Backend . . . 31
9.3.1 Environnement d'exécution : Node.js LTS (Runtime V8 asynchrone) . . . 31
9.3.2 Moteur de sockets collaboratif : y-websocket . . . 31
9.3.3 Passerelle HTTP & Gatekeeper : Express.js / Fastify . . . 31
9.3.4 Kit de développement Cloud : AWS SDK v3 pour JavaScript (@aws-sdk) 31
9.3.5 Moteur de composition documentaire : Typst (ou Puppeteer-core Headless) 32
9.3.6 Gestionnaire de processus système : systemd sous Linux . . . 32
9.4 Justification détaillée des choix technologiques de Persistance . . . 32
9.4.1 Base de données transactionnelle : Amazon DynamoDB (NoSQL clé-valeur) 32
9.4.2 Stockage des artéfacts binaires : Amazon S3 . . . 33
9.4.3 File de messages tampon : Amazon SQS . . . 33
9.5 Serveur Frontal et Reverse-Proxy : Nginx . . . 33
9.6 Matrice de synthèse des choix technologiques . . . 34

Conclusion **35**

Table des figures

Liste des tableaux

4.1 Exigences fonctionnelles du système 5
5.1 Objectifs de performance et métriques cibles 8
6.1 Matrice des droits par acteur et par action 11
8.1 Synthèse des bénéfices architecturaux 28
9.1 Synthèse des composants logiciels et justifications associées 34

Chapitre 1

Présentation générale du projet

1.1 Contexte

Une association publie chaque année une revue numérique rédigée de manière collaborative par plusieurs centaines de rédacteurs. La rédaction d'un même document peut impliquer plusieurs personnes travaillant simultanément. L'association souhaite disposer d'une application permettant de faciliter cette collaboration, de conserver l'historique des modifications et de distribuer les documents sous forme de fichiers PDF.

L'application doit également permettre de contrôler la distribution des documents et de respecter les quotas d'écriture attribués aux rédacteurs.

1.2 Problématique

La mise en place d'un éditeur collaboratif soulève plusieurs difficultés :

- Permettre à plusieurs rédacteurs de modifier simultanément un même document sans perdre leurs modifications.
- Garantir que les participants convergent vers un même contenu.
- Permettre de consulter et de restaurer les versions précédentes d'un document.
- Générer des liens de téléchargement dont le nombre d'utilisations est strictement limité.
- Garantir que les téléchargements correspondent à un état complet et cohérent du document.
- Contrôler les quotas quotidiens d'écriture, y compris lorsque le rédacteur utilise plusieurs appareils.
- Assurer la continuité du service pendant les périodes de forte affluence.

Concevoir un système de rédaction collaborative et de distribution documentaire qui garantit la cohérence des données, le respect des règles de gestion et la maîtrise des accès.

1.3 Objectif général

Concevoir et réaliser une application web permettant à plusieurs rédacteurs de créer, partager et modifier collaborativement des documents textuels, de gérer leurs versions et de distribuer ces documents en PDF au moyen de liens de téléchargement contrôlés.

1.4 Objectifs spécifiques

1. Créer des documents textuels.

2. Partager les documents avec d'autres rédacteurs.
3. Modifier un document simultanément à plusieurs.
4. Visualiser les modifications en quasi-temps réel.
5. Identifier les participants présents et leur position d'écriture.
6. Consulter et restaurer les versions antérieures.
7. Créer des liens de téléchargement avec quota défini.
8. Générer le PDF correspondant à l'état retenu du document.
9. Contrôler les quotas quotidiens d'écriture.
10. Détecter les tentatives de contournement et bloquer temporairement.
11. Permettre à l'association de consulter l'état du système.
12. Notifier le créateur d'un lien à la consommation de sa dernière utilisation.

Chapitre 2

Identification des acteurs

2.1 Le rédacteur

Le rédacteur est un utilisateur qui participe à la production des documents. Il s'identifie en fournissant son nom et son adresse électronique, sans création de compte réel.

2.2 Le visiteur téléchargeur

Le visiteur téléchargeur est anonyme et accède à un document à partir d'un lien de téléchargement.

2.3 L'administrateur de l'association

L'administrateur est chargé de la supervision de l'application. Ses modalités d'authentification sont précisées au paragraphe 5.4.1.

Chapitre 3

Périmètre fonctionnel

3.1 Fonctionnalités incluses

- Gestion des rédacteurs par identification déclarative.
- Création et partage de documents.
- Édition collaborative.
- Présence des participants et visualisation de leur position.
- Gestion et restauration des versions.
- Génération et téléchargement des PDF.
- Gestion des quotas d'écriture.
- Gestion des liens et de leurs quotas.
- Détection des tentatives de contournement.
- Supervision de l'application.

3.2 Limites du périmètre

- Absence de véritables comptes utilisateurs.
- Anonymat des visiteurs téléchargeurs.
- Documents éditables en texte simple.
- Distribution des documents au format PDF.

Chapitre 4

Exigences fonctionnelles

Table 4.1: Exigences fonctionnelles du système

Référence	Exigence
EF01	Créer un document textuel.
EF02	Partager un document avec d'autres rédacteurs.
EF03	Édition simultanée d'un document.
EF04	Propager les modifications en quasi-temps réel.
EF05	Indiquer les participants présents et leurs positions d'écriture.
EF06	Conserver l'historique des versions.
EF07	Consulter et restaurer les versions antérieures.
EF08	Créer un lien de téléchargement avec un quota défini.
EF09	Produire un PDF correspondant à l'état retenu du document au moment de la prise en charge.
EF10	Garantir le respect strict du quota de téléchargement.
EF11	Appliquer le quota quotidien d'écriture de chaque rédacteur.
EF12	Détecter et traiter les tentatives de contournement des quotas.
EF13	Permettre à l'association de superviser l'état de l'application.
EF14	Notifier le créateur d'un lien lorsque sa dernière utilisation autorisée est consommée.
EF15	Informer les visiteurs de l'état de leur demande de téléchargement.

Chapitre 5

Exigences non fonctionnelles et critères mesurables

5.1 Contraintes critiques et majeures du projet

L'énoncé du projet identifie deux contraintes critiques absolues (C1 et C2) dont le non-respect entraîne l'échec immédiat de l'évaluation, complétées par quatre contraintes d'exploitation majeures (C3 à C6) et des contraintes d'infrastructure strictes.

Un lien créé avec un quota de N téléchargements autorisés doit permettre **exactement** N téléchargements réussis : jamais un de plus ($N + 1$), jamais un de moins par perte indue de place, quel que soit le nombre de visiteurs cliquant au même milliseconde.

Critère de vérification : Lors d'un test de charge massif avec plusieurs milliers de requêtes concurrentes sur un lien à quota 500, on doit observer exactement 500 téléchargements accordés, les requêtes surnuméraires rejetées pour épuisement, et strictement zéro surlivraison.

Preuve formelle : Vérification de l'atomicité de l'opération de décrément et de l'invariant `téléchargements_livrés = min(demandes_admissibles, quota)`.

Quel que soit le nombre de rédacteurs intervenant simultanément (jusqu'à 300 rédacteurs actifs), y compris sur le même mot ou le même caractère, aucune modification acceptée par le système ne doit être perdue, et l'ensemble des participants doit converger vers un document strictement identique.

Critère de vérification : Sur un scénario de test injectant des centaines de frappes concurrentes en temps réel, l'état final du document doit être mathématiquement identique sur tous les clients connectés, sans écrasement par la dernière écriture (*Last-Write-Wins* destructif).

Preuve formelle : Utilisation de structures de types de données répliquées sans conflit (CRDT) garantissant la convergence déterministe.

Un lien créé à l'instant T et sollicité à l'instant T_1, après des modifications ultérieures du texte par les rédacteurs, doit impérativement fournir le document complet et cohérent tel qu'il est à l'instant T_1, sans nécessiter la génération ni la diffusion d'un nouveau lien.

Critère de vérification : Une modification apportée à $T_1 - \epsilon$ doit figurer dans le PDF généré pour le visiteur admis à T_1.

Chaque rédacteur dispose d'un crédit d'écriture strictement plafonné à **10 000 caractères par jour**, rechargé quotidiennement. Ce plafond s'applique à l'identité déclarée du rédacteur de manière centralisée, y compris s'il ouvre plusieurs fenêtres ou se connecte depuis plusieurs appareils distincts simultanément. Toute tentative d'acharnement contre ces limites doit entraîner un blocage temporaire automatique.

Critère de vérification : Injection parallèle de requêtes d'insertion depuis plusieurs sessions simultanées ; le cumul des caractères acceptés sur la journée ne doit jamais dépasser 10 000.

À l'heure précise de parution du journal, le système doit absorber un afflux de **15 000 visiteurs**, dont **80 %** surviennent dans la première minute (soit un débit moyen d'environ 200 requêtes par seconde, avec des pointes instantanées pouvant atteindre 500 à 800 requêtes par seconde), tout en maintenant l'édition active pour 300 rédacteurs. L'application ne doit ni planter, ni abandonner de requêtes silencieusement : chaque visiteur doit recevoir immédiatement une réponse explicite (téléchargement immédiat, mise en attente de compilation, ou épuisement).

Critère de vérification : Test d'injection de 15 000 requêtes sur 60 secondes ; absence d'erreurs 502/504 et temps de réponse rapide pour le refus des places épuisées.

Dès que le dernier exemplaire (N-ième) est attribué, le système doit immédiatement orienter les visiteurs suivants vers un état d'épuisement clair, sans déclencher de calculs ni de compilations PDF inutiles. Le créateur du lien doit être immédiatement notifié par e-mail de la distribution du dernier exemplaire.

Critère de vérification : Temps de réponse inférieur à 20 ms pour le renvoi du statut épuisé après attribution de la dernière place, et réception d'un unique message électronique de notification.

5.2 Contraintes d'infrastructure et règles d'évaluation

Le sujet impose un ensemble de règles strictes régissant l'hébergement et la conception de la solution :

- **Compte AWS et plan d'accès :** L'infrastructure est déployée sur un compte AWS personnel, optimisée pour le plan gratuit (*Free Tier*) ou couverte par les crédits pédagogiques disponibles (enveloppe de 100 $ USD), garantissant un coût réel de 0,00 $ pour l'étudiant.
- **Région unique :** L'ensemble des composants doit résider au sein d'une seule et même région AWS (par exemple `eu-west-3` Paris ou `eu-north-1` Stockholm).
- **Stockage des documents :** L'énoncé prescrit explicitement que les fichiers PDF finaux doivent être conservés dans le service de stockage objet Amazon S3.
- **Sécurité et gestion des identifiants (IAM) :** Aucune clé d'accès statique (de type `AKIA...`) ne doit apparaître dans le code source ou dans le dépôt Git. Les composants doivent s'exécuter sous le principe du moindre privilège via des rôles d'instance sécurisés (IMDSv2).
- **Délai de réalisation court (Un week-end) :** Le projet étant réalisé du vendredi soir au lundi matin, l'architecture doit privilégier des composants fiables, maintenables et simples à

expliquer et tester, plutôt qu'une sur-complexification logicielle.

5.3 Disponibilité et capacité : objectifs chiffrés mesurables

Table 5.1: Objectifs de performance et métriques cibles

Indicateur	Objectif cible	Condition
Taux de succès de réponse	100 % des requêtes traitées	Pic de 15 000 visiteurs
Temps de réponse (refus quota épuisé)	$P_{95} < 50$ ms	Pendant le pic
Latence de synchronisation collaborative	$P_{95} < 200$ ms	300 rédacteurs actifs
Dépassement de quota autorisé	Strictement 0	Test de concurrence
Perte de modification acceptée	Strictement 0	Test de convergence
Dépassement crédit de 10 000 caractères	Strictement 0	Multi-sessions

5.4 Sécurité et gestion des accès

- Application stricte du moindre privilège sur chaque ressource AWS.
- Absence totale d'identifiants permanents dans le code ou sur l'instance.
- Isolation des données : accès au bucket S3 restreint au rôle de l'application.
- Chiffrement des communications en transit via HTTPS et WebSockets sécurisés (WSS).

5.4.1 Authentification de l'administrateur

L'authentification de l'administrateur s'effectue via un secret fort stocké dans les variables d'environnement système (ou AWS Systems Manager Parameter Store), vérifié côté serveur avec protection contre les attaques par force brute.

Chapitre 6

Principes métier fondamentaux

6.1 Principe de cohérence documentaire

Un téléchargement doit correspondre à un état identifiable du document. Une fois cet état retenu pour une demande, le contenu utilisé pour produire le PDF ne doit pas changer pendant sa génération. Une demande ultérieure peut correspondre à un état plus récent.

Traduction : l'état est identifié par un numéro de version, figé au moment de la transition *reçue* → *autorisée* de la demande (cf. section 7.4.1).

6.2 Principe de validité du lien

Un lien de téléchargement référence un document et possède un quota d'utilisation. Il n'est pas lié à une copie figée du contenu au moment de sa création. Tant qu'il reste valide et qu'une utilisation est autorisée, il permet de télécharger l'état du document retenu pour la nouvelle demande.

6.3 Principe de respect strict du quota

Le nombre de téléchargements effectivement livrés ne doit jamais dépasser le quota défini, et doit l'atteindre lorsque la demande le permet. Un lien à 3 téléchargements doit permettre exactement 3 téléchargements réussis, jamais 2 par perte de place.

Deux quantités distinctes doivent être définies sans ambiguïté :

- **Place réservée** : une place est réservée lors de la transition *reçue* → *autorisée* de la demande de téléchargement.
- **Téléchargement livré** : un téléchargement est compté comme livré lorsque le PDF est effectivement remis au visiteur.

Choix retenu : **la place est réservée à l'admission** (transition *reçue* → *autorisée*). Si la génération du PDF échoue, ou si le visiteur n'aboutit pas, la place est **rendue** (transition *autorisée* → *échouée* → *libérée*). Ainsi un lien à quota 3 permet bien 3 téléchargements réussis en cas de panne transitoire, tout en garantissant qu'aucun 4e téléchargement ne peut être admis tant que les 3 places ne sont pas rendues ou livrées.

Le compteur exposé à l'administrateur est le nombre de places *disponibles* (quota moins places réservées et moins téléchargements livrés).

6.4 Principe de conservation des modifications

Toute modification acceptée doit être conservée. Lorsque plusieurs rédacteurs interviennent simultanément, le système doit préserver leurs modifications et garantir la convergence du document. Ce principe est critique (cf. contrainte C2).

6.5 Principe de limitation de l'écriture

Chaque rédacteur bénéficie d'un crédit quotidien de **10 000 caractères**, rechargé chaque jour. Lorsque son crédit est épuisé, il ne peut plus écrire jusqu'au prochain rechargement, même en changeant d'appareil.

6.5.1 Précisions sur le crédit d'écriture

Définitions retenues pour lever toute ambiguïté :

- **Unité comptée** : caractères *insérés*. Les suppressions et les remplacements ne sont pas comptés, seules les insertions nettes le sont.
- **Collage** : un collage de n caractères consomme n unités. Un collage refusé faute de crédit n'insère rien et ne consomme rien.
- **Fuseau horaire** : le « jour » est défini en UTC, rechargement à 00:00 UTC.
- **Clé d'identification** : le couple (nom, e-mail) fourni à l'entrée est la clé de comptage. C'est aussi la limite assumée : un rédacteur qui change d'identité contourne la limite.
- **Multi-appareils** : le crédit est porté par l'identité, non par l'appareil ; il est donc partagé entre appareils.

6.6 Principe de blocage en cas de tentative de contournement

Seuils et durées retenus, ajustables après tests :

- **Clé de blocage** : couple (nom, e-mail).
- **Seuil** : blocage déclenché au-delà de 3 tentatives d'écriture refusées pour crédit épuisé dans une fenêtre de 10 minutes.
- **Durée** : blocage temporaire de 24 heures.
- **Limite assumée** : l'identification étant déclarative, changer d'identité contourne le blocage. C'est un choix de conception cohérent avec l'absence de comptes réels, documenté comme tel.

6.7 Principe de gestion des versions et de l'historique

Le sujet parle de « versions précédentes », le présent rapport parle d'« historique des modifications ». Ce ne sont pas la même granularité. Choix retenu :

- **Granularité** : une version est créée à chaque sauvegarde explicite et à chaque fin de session d'édition (regroupement des modifications d'une même session).
- **Conservation de l'historique** : restaurer une version ne détruit pas l'historique. La restauration *crée une nouvelle version* dont le contenu est celui de la version restaurée.
- **Crédit d'écriture** : restaurer ne consomme pas de crédit d'écriture (ce n'est pas une écriture au sens de la saisie).

6.8 Principe de droits d'accès

Le sujet mentionne « selon ses droits » sans les définir. Le tableau 6.1 retient la matrice suivante.

Table 6.1: Matrice des droits par acteur et par action

Action	Créateur	Collaborateu	Visiteur	Admin
Lire le document	✓	✓	—	✓
Écrire dans le document	✓	✓	—	—
Partager le document	✓	—	—	—
Consulter l'historique	✓	✓	—	✓
Restaurer une version	✓	—	—	—
Créer un lien de téléchargement	✓	—	—	—
Télécharger via un lien	—	—	✓	—
Superviser l'application	—	—	—	✓

6.9 Principe de la notification au créateur

Un seul e-mail est envoyé au créateur du lien lorsqu'il ne reste plus aucune place disponible. L'envoi est déclenché par la transaction qui consomme la dernière place, dans une section critique, afin qu'en cas de concurrence un seul envoi soit effectué.

6.10 Principe des paramètres du lien

Un lien possède un quota (nombre maximal de téléchargements) et une date d'expiration optionnelle. En l'absence de date, il n'expire pas. Il devient inutilisable dès que le quota est atteint.

Chapitre 7

Analyse et conception

Ce chapitre présente la modélisation du système au moyen de diagrammes UML : vue fonctionnelle (cas d'utilisation), vue structurelle (classes), vue dynamique (activités et états-transitions). Les diagrammes eux-mêmes sont fournis dans les annexes graphiques ; ce chapitre en donne les descriptions textuelles et les commentaires.

7.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation du système met en relation trois acteurs (rédacteur, visiteur téléchargeur, administrateur) et les cas d'utilisation suivants :

- Pour le rédacteur : s'identifier, créer un document, inviter des collaborateurs, modifier un document, consulter les versions, restaurer une version, créer un lien de téléchargement.
- Pour le visiteur : télécharger un journal, consulter l'état de la demande de téléchargement.
- Pour l'administrateur : consulter les documents ouverts, consulter les liens de téléchargement, consulter les rédacteurs bloqués.

Le cas *Créer un lien de téléchargement* inclut la consultation des versions pour garantir qu'un état cohérent est référencé. Le cas *Télécharger un journal* inclut la consultation de l'état de la demande pour informer le visiteur. Le cas *Restaurer une version* étend *Consulter les versions*.

7.1.1 Descriptions textuelles des cas d'utilisation

Les descriptions ci-dessous reprennent, pour chaque cas d'utilisation identifié, l'acteur principal, les préconditions, le scénario nominal, les scénarios alternatifs ou exceptions, et les postconditions.

UC01 — S'identifier

Acteur principal : Rédacteur.

Objectif : Permettre à un rédacteur de fournir son identité afin d'utiliser les fonctionnalités de l'application.

Préconditions : L'application est accessible.

Scénario nominal :

1. Le rédacteur accède à l'application.
2. Le système demande son nom et son adresse e-mail.
3. Le rédacteur renseigne son nom et son adresse e-mail.
4. Le système vérifie la validité des informations fournies.
5. Le système associe l'identité fournie au rédacteur concerné, selon le mécanisme d'identification retenu.
6. Le système permet au rédacteur d'accéder aux fonctionnalités autorisées.

Scénarios alternatifs et exceptions :

- **A1** : Le nom ou l'adresse e-mail est manquant. Le système demande de compléter les informations.
- **A2** : L'adresse e-mail est invalide. Le système demande une correction.
- **A3** : Une erreur survient pendant l'identification. Le système informe le rédacteur.

Postconditions : L'identité déclarée du rédacteur est disponible ; le rédacteur peut accéder aux fonctionnalités correspondant à ses droits.

Remarque : Le sujet précise qu'il n'y a pas de véritables comptes rédacteurs. Ce cas décrit donc la fourniture et la reconnaissance de l'identité déclarée, et non une authentification par mot de passe.

UC02 — Créer un document

Acteur principal : Rédacteur.

Objectif : Permettre à un rédacteur de créer un nouveau document textuel dans l'application.

Préconditions :

- Le rédacteur a renseigné son nom et son adresse e-mail.
- Le rédacteur n'est pas temporairement bloqué.

Scénario nominal :

1. Le rédacteur demande la création d'un document.
2. Le système vérifie que le rédacteur peut effectuer cette opération.
3. Le système vérifie que le rédacteur n'est pas bloqué.
4. Le système crée un nouveau document vide.
5. Le système associe le document à son créateur.
6. Le système initialise les informations du document.
7. Le système affiche le document dans l'éditeur.
8. Le rédacteur peut commencer à écrire.

Scénarios alternatifs et exceptions :

- **A1** : Le rédacteur est temporairement bloqué. Le système refuse la création si cette opération est interdite aux rédacteurs bloqués.
- **A2** : Une erreur survient pendant la création. Le système informe le rédacteur de l'échec.

Postconditions : Le document est créé et associé à son créateur ; il est disponible pour les opérations autorisées.

UC03 — Inviter des collaborateurs

Acteur principal : Rédacteur.

Objectif : Permettre au propriétaire d'un document de donner accès à d'autres rédacteurs afin qu'ils puissent collaborer.

Préconditions :

- Le document existe.
- Le rédacteur est le propriétaire du document.
- Le rédacteur n'est pas bloqué, si le blocage interdit cette opération.

Scénario nominal :

1. Le rédacteur ouvre le document concerné.
2. Le rédacteur sélectionne la fonctionnalité de partage.
3. Le système vérifie les droits du rédacteur.
4. Le rédacteur renseigne les informations nécessaires pour identifier le collaborateur, notamment son adresse e-mail.
5. Le système vérifie les informations fournies.
6. Le système enregistre l'accès du collaborateur au document.
7. Le système confirme le partage au rédacteur.

Scénarios alternatifs et exceptions :

- **A1** : Le rédacteur n'est pas propriétaire du document. Le système refuse l'opération.
- **A2** : Les informations du collaborateur sont invalides. Le système demande leur correction.
- **A3** : Le collaborateur possède déjà un accès. Le système informe le rédacteur.
- **A4** : Une erreur survient pendant le partage. Le système signale l'échec.

Postconditions : Le collaborateur dispose d'un accès au document conformément aux droits définis ; le système conserve les informations nécessaires à la gestion du partage.

UC04 — Modifier un document

Acteur principal : Rédacteur.

Acteurs secondaires : Autres rédacteurs collaborateurs.

Objectif : Permettre à un rédacteur autorisé de modifier un document, éventuellement en collaboration avec d'autres rédacteurs.

Préconditions :

- Le rédacteur est identifié.
- Le document existe.
- Le rédacteur est propriétaire du document ou dispose d'un accès collaborateur.
- Le rédacteur n'est pas bloqué.

Scénario nominal :

1. Le rédacteur ouvre le document.
2. Le système vérifie ses droits d'accès.
3. Le système vérifie que le rédacteur peut écrire.
4. Le système vérifie que son crédit d'écriture est disponible.
5. Le système affiche le contenu actuel du document.
6. Le système indique les collaborateurs présents et leurs positions d'écriture.
7. Le rédacteur saisit ou modifie du texte.
8. Le système contrôle la quantité de caractères que le rédacteur peut encore écrire.
9. Le système accepte les modifications respectant les règles de crédit.
10. Le système applique les modifications sans perdre celles des autres collaborateurs.
11. Le système propage les modifications aux collaborateurs en quasi-temps réel.
12. Le système met à jour le crédit d'écriture du rédacteur.

13. Le système conserve les modifications acceptées et l'historique nécessaire.

Scénarios alternatifs et exceptions :

- **A1** : Le rédacteur ne possède pas les droits nécessaires. Le système refuse l'accès en écriture.
- **A2** : Le rédacteur est bloqué. Le système lui interdit d'écrire.
- **A3** : Le crédit est épuisé. Le système refuse les nouvelles écritures jusqu'à la recharge du crédit.
- **A4** : La saisie dépasse le crédit restant. Le système limite ou refuse la partie excédentaire, selon la stratégie d'édition retenue.
- **A5** : Plusieurs rédacteurs modifient simultanément le document. Le système coordonne les modifications afin de préserver toutes les modifications acceptées.
- **A6** : Une tentative de contournement des limites est détectée. Le système applique le mécanisme de détection et de blocage temporaire prévu.
- **A7** : Une erreur de connexion survient. Le système informe le rédacteur et applique la stratégie de récupération prévue.

Postconditions :

- Les modifications acceptées sont conservées.
- Les crédits sont mis à jour sans dépassement.
- Les collaborateurs convergent vers un même état du document.

UC05 — Consulter les versions

Acteur principal : Rédacteur.

Objectif : Permettre à un rédacteur autorisé de consulter l'historique des versions d'un document.

Préconditions :

- Le document existe.
- Le rédacteur dispose des droits nécessaires pour consulter le document.

Scénario nominal :

1. Le rédacteur sélectionne le document concerné.
2. Le rédacteur demande à consulter les versions.
3. Le système vérifie les droits d'accès du rédacteur.
4. Le système récupère l'historique des versions disponibles.
5. Le système affiche la liste des versions avec leurs informations pertinentes : numéro, date et auteur, selon les données conservées.
6. Le rédacteur sélectionne une version.
7. Le système affiche le contenu de la version sélectionnée.
8. Le rédacteur consulte la version.

Scénarios alternatifs et exceptions :

- **A1** : Le document ne possède pas de version historique disponible. Le système informe le rédacteur.
- **A2** : Le rédacteur ne dispose pas des droits nécessaires. Le système refuse l'accès.

- **A3** : La version sélectionnée est indisponible. Le système informe le rédacteur.
- **A4** : Une erreur survient lors du chargement de l'historique. Le système signale l'échec.

Postconditions : Le rédacteur a pu consulter les versions accessibles ; le contenu courant n'est pas modifié par la simple consultation.

Remarque : La restauration d'une version constitue un cas d'utilisation distinct (UC06).

UC06 — Restaurer une version

Acteur principal : Rédacteur.

Objectif : Permettre à un rédacteur autorisé de restaurer une version antérieure d'un document.

Préconditions :

- Le document existe.
- Le rédacteur dispose des droits nécessaires.
- Le document possède au moins une version antérieure consultable.

Scénario nominal :

1. Le rédacteur ouvre le document.
2. Le rédacteur demande la consultation de l'historique.
3. Le système affiche les versions disponibles.
4. Le rédacteur sélectionne une version.
5. Le système affiche le contenu de la version sélectionnée.
6. Le rédacteur demande la restauration.
7. Le système vérifie les droits du rédacteur.
8. Le système vérifie les conditions de restauration.
9. Le système restaure le contenu correspondant à la version sélectionnée.
10. Le système conserve l'historique et enregistre cette restauration.
11. Le système propage le nouvel état du document aux collaborateurs.

Scénarios alternatifs et exceptions :

- **A1** : Aucune version antérieure n'est disponible. Le système informe le rédacteur.
- **A2** : La version sélectionnée n'existe plus ou n'est pas accessible. Le système refuse la restauration.
- **A3** : Le rédacteur ne dispose pas des droits nécessaires. Le système refuse l'opération.
- **A4** : Des modifications simultanées sont en cours. Le système coordonne la restauration avec les autres modifications pour éviter toute perte de données.
- **A5** : Une erreur survient pendant la restauration. Le système signale l'échec sans présenter une restauration incomplète comme réussie.

Postconditions : Le contenu du document correspond à la version restaurée, sous réserve de la stratégie de restauration retenue ; l'historique est préservé ; les collaborateurs peuvent consulter l'état restauré.

UC07 — Créer un lien de téléchargement

Acteur principal : Rédacteur.

Objectif : Permettre à un rédacteur autorisé de créer un lien de téléchargement pour un document, avec un quota défini.

Préconditions :

- Le document existe.
- Le rédacteur dispose des droits nécessaires.
- Le rédacteur n'est pas bloqué, si le blocage interdit cette opération.

Scénario nominal :

1. Le rédacteur ouvre le document concerné.
2. Le rédacteur demande la création d'un lien de téléchargement.
3. Le système vérifie ses droits.
4. Le système demande le nombre de téléchargements autorisés.
5. Le rédacteur saisit le quota souhaité.
6. Le système vérifie que le quota est valide et strictement positif.
7. Le système crée un lien unique associé au document.
8. Le système initialise le quota total et le nombre de téléchargements restants.
9. Le système active le lien.
10. Le système communique le lien au rédacteur.

Scénarios alternatifs et exceptions :

- **A1** : Le quota fourni est invalide. Le système demande une nouvelle valeur.
- **A2** : Le rédacteur ne dispose pas des droits nécessaires. Le système refuse la création.
- **A3** : Une erreur survient pendant la création. Le système informe le rédacteur de l'échec.

Postconditions : Un lien de téléchargement est créé, associé au document, avec quota initialisé ; il permet de télécharger le document dans la limite autorisée.

UC08 — Télécharger un journal

Acteur principal : Visiteur.

Objectif : Permettre à un visiteur anonyme de télécharger le document au format PDF, dans la limite du quota du lien.

Préconditions :

- Le visiteur possède un lien de téléchargement.
- Le lien correspond à un lien reconnu par le système.

Scénario nominal :

1. Le visiteur ouvre le lien de téléchargement.
2. Le système vérifie l'existence et la validité du lien.
3. Le système vérifie l'état du lien et son quota disponible.
4. Le système vérifie qu'une place de téléchargement peut être attribuée.
5. Le système réserve atomiquement une place dans le quota afin de prévenir tout dépassement en cas de demandes simultanées.
6. Le système indique au visiteur que sa demande est en cours de traitement.
7. Le système obtient un état cohérent du document au moment de la prise en charge du téléchargement.
8. Le système génère le PDF correspondant à cet état.
9. Le système met le fichier à disposition du visiteur.

10. Le système indique que le téléchargement est terminé ou fournit les informations nécessaires à sa récupération.
11. Le système finalise le traitement du téléchargement.
12. Si le dernier téléchargement autorisé est consommé, le système informe le créateur du lien par e-mail.

Scénarios alternatifs et exceptions :

- **A1** : Le lien est invalide. Le système informe le visiteur.
- **A2** : Le quota est épuisé. Le système indique clairement que le journal n'est plus disponible.
- **A3** : Le visiteur arrive pendant un pic de charge. Le système lui indique si son téléchargement est en cours, en attente ou si le quota est épuisé.
- **A4** : Le quota est consommé pendant le traitement d'autres demandes. Le système refuse les nouvelles demandes non réservées.
- **A5** : Le PDF ne peut pas être généré. Le système applique une politique de traitement des échecs cohérente avec la réservation du quota.
- **A6** : Le téléchargement échoue pendant la transmission. Le système applique la politique prévue pour le quota et informe le visiteur.
- **A7** : Le dernier exemplaire vient d'être distribué. Le système informe rapidement les visiteurs encore en attente que le quota est épuisé.

Postconditions :

- Le téléchargement autorisé est traité conformément aux règles de quota.
- Le quota ne dépasse jamais le nombre autorisé.
- Le visiteur reçoit une information sur l'état de sa demande.
- Le document téléchargé correspond à un état complet et cohérent.

UC09 — Consulter l'état de la demande de téléchargement

Acteur principal : Visiteur.

Objectif : Permettre au visiteur de connaître l'état de sa demande de téléchargement.

Préconditions :

- Le visiteur a initié une demande de téléchargement.
- Le système dispose d'un moyen permettant de retrouver cette demande, par exemple un identifiant ou un mécanisme de suivi.

Scénario nominal :

1. Le visiteur accède à l'interface de suivi de sa demande.
2. Le visiteur fournit ou utilise les informations de suivi disponibles.
3. Le système recherche la demande correspondante.
4. Le système détermine l'état actuel de la demande.
5. Le système affiche l'état au visiteur.

Scénarios alternatifs et exceptions :

- **A1** : La demande est en attente. Le système indique que le visiteur doit patienter.
- **A2** : La demande est en cours de traitement. Le système informe le visiteur.

- **A3** : La demande est terminée. Le système indique que le téléchargement est disponible ou terminé.
- **A4** : Le quota est épuisé avant la prise en charge. Le système informe le visiteur que le téléchargement n'est plus disponible.
- **A5** : La demande est refusée. Le système indique le motif lorsque celui-ci peut être communiqué.
- **A6** : La demande est introuvable ou l'identifiant de suivi est invalide. Le système informe le visiteur.

Postconditions : Le visiteur connaît l'état de sa demande ; aucune modification du quota n'est effectuée par la simple consultation.

UC10 — Consulter les documents ouverts

Acteur principal : Administrateur.

Objectif : Permettre à l'administrateur de superviser les documents actuellement ouverts dans l'application.

Préconditions :

- L'administrateur est identifié.
- L'administrateur dispose des droits de supervision.

Scénario nominal :

1. L'administrateur accède à l'espace de supervision.
2. Il sélectionne la fonctionnalité de consultation des documents ouverts.
3. Le système vérifie ses droits.
4. Le système recherche les documents actuellement ouverts.
5. Le système récupère les informations de supervision disponibles.
6. Le système affiche les documents ouverts.
7. L'administrateur consulte les informations affichées, notamment les documents concernés et, si disponible, les rédacteurs présents.
8. L'administrateur termine la consultation.

Scénarios alternatifs et exceptions :

- **A1** : Aucun document n'est actuellement ouvert. Le système affiche un message approprié.
- **A2** : L'administrateur ne possède pas les droits nécessaires. Le système refuse l'accès.
- **A3** : Un document se ferme pendant la consultation. Le système actualise son état selon le mécanisme de supervision retenu.
- **A4** : Une erreur survient lors de la récupération des données. Le système informe l'administrateur.

Postconditions : L'administrateur a pu consulter l'état des documents ouverts ; aucune modification du contenu n'est effectuée par la simple consultation.

Remarque : Les informations de présence doivent correspondre à l'état réel des sessions. Le système doit prévoir comment traiter les sessions interrompues ou les déconnexions non signalées.

UC11 — Consulter les liens de téléchargement

Acteur principal : Administrateur.

Objectif : Permettre à l'administrateur de consulter les liens de téléchargement actifs et leurs quotas restants.

Préconditions :

- L'administrateur est identifié.
- L'administrateur dispose des droits de supervision.

Scénario nominal :

1. L'administrateur accède à l'espace de supervision.
2. Il sélectionne la fonctionnalité de consultation des liens de téléchargement.
3. Le système vérifie ses droits.
4. Le système récupère les liens de téléchargement à superviser.
5. Le système récupère les informations associées à chaque lien.
6. Le système affiche les informations pertinentes, notamment : le document concerné, l'état du lien, le quota initial, le nombre de téléchargements consommés, le nombre de téléchargements restants.
7. L'administrateur consulte les informations.

Scénarios alternatifs et exceptions :

- **A1** : Aucun lien n'est disponible. Le système affiche un message approprié.
- **A2** : L'administrateur ne possède pas les droits nécessaires. Le système refuse l'accès.
- **A3** : Certains liens sont épuisés ou désactivés. Le système affiche leur état respectif.
- **A4** : Une erreur survient lors du chargement des données. Le système informe l'administrateur.

Postconditions : L'administrateur a pu consulter les informations disponibles sur les liens ; aucune modification du quota ou de l'état des liens n'est effectuée par la simple consultation.

Remarque : La consultation porte sur les liens et leurs quotas. Une opération de modification ou de désactivation des liens nécessiterait un cas d'utilisation supplémentaire si elle est retenue.

UC12 — Consulter les rédacteurs bloqués

Acteur principal : Administrateur.

Objectif : Permettre à l'administrateur de consulter la liste des rédacteurs temporairement bloqués par le système.

Préconditions :

- L'administrateur est identifié et autorisé à accéder à la fonctionnalité de supervision.

Scénario nominal :

1. L'administrateur accède à l'espace de supervision.
2. Il sélectionne la fonctionnalité de consultation des rédacteurs bloqués.
3. Le système vérifie ses droits.
4. Le système recherche les rédacteurs actuellement bloqués.
5. Le système récupère les informations nécessaires à leur identification et à leur suivi.
6. Le système affiche la liste des rédacteurs bloqués.
7. L'administrateur consulte les informations disponibles.

Scénarios alternatifs et exceptions :

- **A1** : Aucun rédacteur n'est actuellement bloqué. Le système affiche une liste vide ou un message approprié.
- **A2** : L'administrateur ne possède pas les droits nécessaires. Le système refuse l'accès.
- **A3** : Une erreur survient lors de la récupération des informations. Le système informe l'administrateur.

Postconditions : L'administrateur a pu consulter la liste des rédacteurs bloqués ; aucune modification du statut des rédacteurs n'est effectuée par la simple consultation.

Remarque : Le déblocage d'un rédacteur n'est pas décrit comme un cas d'utilisation distinct dans le diagramme présenté. Il ne faut donc pas le considérer comme une fonctionnalité acquise sans validation.

7.2 Diagramme de classes

Le diagramme de classes du système comprend les entités principales suivantes et leurs relations :

- **Rédacteur** : id, nom, email, crédit restant ; participe à un ou plusieurs documents ; possède un quota d'écriture par jour ; peut être bloqué.
- **Document** : id, titre, contenu, version active ; possède un ensemble de versions (au moins une) ; peut générer plusieurs liens de téléchargement.
- **Version** : id, numéro, contenu, horodatage ; appartient à un document ; peut être référencée par un téléchargement.
- **LienTéléchargement** : id, quota, téléchargements consommés, créateur ; associé à un document ; concerne plusieurs téléchargements.
- **Téléchargement** : id, lien, version référencée, état, date de demande ; traite la génération du PDF.
- **QuotaÉcriture** : rédacteur, date, caractères utilisés, limite ; consommé par les modifications.
- **Blocage** : id, rédacteur, motif, dates de début et de fin.

La multiplicité `1..*` entre `Document` et `Version` traduit le fait qu'un document possède au moins une version et en accumule à chaque sauvegarde. Le lien de téléchargement n'est pas lié à une version figée : c'est la `Téléchargement` qui référence la version au moment de la demande, garantissant le principe de validité du lien.

7.3 Diagrammes d'activités

7.3.1 Distribution d'un document via un lien

Le processus de distribution d'un document via un lien se déroule ainsi :

1. Le visiteur demande le téléchargement.
2. Le système vérifie si le quota est disponible. Si non, il refuse avec la mention « quota épuisé ».

3. Si oui, le système réserve atomiquement une place.
4. Le système fige l'état du document (référence à une version).
5. Le système génère le PDF.
6. Si la génération réussit, le système livre le PDF au visiteur. Si la génération échoue, la place est rendue et le visiteur est notifié.
7. Après livraison, le système vérifie si la place consommée était la dernière. Si oui, il notifie le créateur par e-mail.

La place est réservée *avant* la génération du PDF (réservation atomique). Si la génération échoue, la place est rendue, ce qui garantit qu'un lien à quota N permet bien N téléchargements réussis, même en cas de panne transitoire, sans jamais dépasser N.

7.3.2 Modification d'un document

Le processus de modification d'un document par un rédacteur se déroule ainsi :

1. Le rédacteur ouvre le document.
2. Le système vérifie ses droits d'écriture. Si non, il refuse.
3. Le système vérifie que le rédacteur n'est pas bloqué. Si oui, il refuse.
4. Le système vérifie que le crédit est disponible. Si non, il refuse les nouvelles écritures.
5. Le rédacteur saisit ou modifie du texte.
6. Le système fusionne sans perte avec l'état courant.
7. Le système propage la modification aux collaborateurs.
8. Le système met à jour le crédit du rédacteur.
9. Le système conserve la modification et l'historique.

7.4 Diagrammes d'états-transitions

7.4.1 Cycle de vie d'une demande de téléchargement

Une demande de téléchargement passe par les états suivants :

- **Reçue** : la demande est enregistrée, avant décision.
- **En attente** : la demande est placée en file si la capacité ou les places disponibles ne permettent pas une admission immédiate.
- **Autorisée** : une place a été réservée atomiquement. C'est à cette transition que la place est consommée au sens de la réservation.
- **Refusée « épuisé »** : aucune place n'est disponible. Cet état est définitif pour un lien donné.
- **PDF prêt** : le PDF a été généré pour l'état figé du document.
- **Terminée** : le PDF a été livré. La place est définitivement consommée au sens de la livraison.
- **Échouée** : la génération du PDF a échoué ou le visiteur n'a pas abouti. La place est rendue.

La place est **réservée** à la transition *Reçue* → *Autorisée*, ce qui empêche tout dépassement en cas de concurrence. Elle est **livrée** à la transition *PDF prêt* → *Terminée*, et **rendue** à la transition *PDF prêt* → *Échouée*. Aucune demande ne peut passer de *Refusée* à un autre état : le quota épuisé est définitif pour un lien donné.

7.4.2 Cycle de vie d'un document et des versions

Un document passe par les états suivants : *Créé*, *En édition*, *Version sauvegardée*, *Version restaurée*. Les transitions sont :

- *Créé* → *En édition* : à l'ouverture pour édition.
- *En édition* → *Version sauvegardée* : à chaque sauvegarde explicite ou fin de session.
- *Version sauvegardée* → *En édition* : à la reprise d'édition.
- *En édition* → *Version restaurée* : à la demande de restauration.
- *Version restaurée* → *Version sauvegardée* : la restauration crée une nouvelle version.

Restaurer ne détruit pas l'historique : cela crée une nouvelle version dont le contenu est celui de la version restaurée (cf. section 5.4.1 et les hypothèses sur la gestion des versions).

Chapitre 8

Architecture Cloud AWS

8.1 Présentation générale de l'architecture retenue

Pour répondre aux contraintes du projet tout en garantissant une faisabilité totale dans les délais impartis, l'infrastructure cloud retenue repose sur une **architecture à deux voies spécialisées**, déployée dans une région unique (`eu-west-3` Paris ou `eu-north-1` Stockholm) et strictement contenue dans le périmètre de l'offre gratuite et des crédits alloués :

1. **La voie d'édition collaborative (temps réel persistant) :** dédiée aux 300 rédacteurs actifs, caractérisée par des connexions longues et de multiples échanges de petits deltas textuels avec une latence réseau minimale.
2. **La voie de distribution documentaire (haute performance sans état) :** dédiée à l'absorption du pic de 15 000 visiteurs, caractérisée par un filtrage immédiat des accès, une réservation atomique stricte des places autorisées, un traitement asynchrone de la compilation et une livraison déléguée des flux binaires.

Cette conception garantit un cloisonnement strict des ressources : un afflux massif de visiteurs sur les liens de téléchargement ne dégrade en aucun cas la réactivité de l'éditeur partagé pour les rédacteurs en cours de travail.

8.2 Justification et rôle de chaque service AWS

Chaque service utilisé a été sélectionné pour ses propriétés intrinsèques et configuré selon un rôle précis, aligné avec les bonnes pratiques d'ingénierie cloud.

8.2.1 Amazon EC2 : hébergement de l'application et de la collaboration temps réel

Configuration : Une instance de type `t3.micro` sous Amazon Linux 2023, protégée par un groupe de sécurité (*Security Group*) restreint.

Rôle au sein de l'architecture :

- **Reverse-Proxy frontal Nginx :** Écoute sur les ports 80 (redirection HTTPS) et 443. Il assure la terminaison TLS/SSL, la compression des flux statiques, et route le trafic soit vers l'API HTTP locale, soit vers le serveur de sockets avec gestion des en-têtes de mise à niveau de protocole (`Upgrade: websocket`).
- **Moteur de collaboration temps réel (Node.js / y-websocket) :** Maintient les connexions WebSockets persistantes des rédacteurs. Une instance de cette classe prend en charge sans difficulté plusieurs milliers de sockets en mémoire vive avec une consommation inférieure à 200 Mo de mémoire et une latence de propagation inter-clients inférieure à 5 ms.
- **Supervision locale systemd :** L'application et le serveur de collaboration sont exécutés sous forme de services gérés par `systemd` avec la directive `Restart=always`, assurant la reprise automatique sans intervention humaine en cas d'anomalie transitoire.

Justification de ce choix : L'édition collaborative temps réel nécessite le maintien d'un état en mémoire et une diffusion instantanée aux pairs connectés. L'utilisation d'un serveur d'application persistant sur EC2 évite tout surcoût d'invocation par message et permet l'exploitation directe des protocoles de synchronisation CRDT standards de l'industrie.

8.2.2 Amazon DynamoDB : persistance d'état et décompte atomique

Configuration : Table unique (*Single-Table Design*) provisionnée en mode de capacité à la demande (*On-Demand*), minimisant les coûts et absorbant instantanément les pics de débit.

Rôle au sein de l'architecture :

- **Résolution formelle de la contrainte C1 (Exactement N téléchargements) :** DynamoDB stocke l'élément représentant le lien de téléchargement avec son solde de places disponibles. L'attribution d'une place s'effectue via une opération atomique d'écriture conditionnelle :

$$\textbf{SET} \quad \text{places_restantes} = \text{places_restantes} - 1 \quad \textbf{WHERE} \quad \text{places_restantes} > 0 \tag{8.1}$$

  Cette instruction est exécutée de façon strictement sérialisée au cœur de la couche de réplication de stockage d'AWS. Même sous un afflux de 500 requêtes frappant la même milliseconde, les accès concurrents sont arbitrés sans verrou bloquant applicatif. Lorsque le compteur atteint zéro, toute requête suivante déclenche immédiatement l'exception `ConditionalCheckFailedException`.
- **Application stricte du crédit d'écriture (C4) :** Le cumul des caractères insérés par chaque rédacteur est incrémenté de façon atomique sur un enregistrement dont la clé intègre la date du jour (`USER#<email>`, `CREDIT#<YYYY-MM-DD>`), garantissant un décompte unifié même en cas de sessions multi-appareils simultanées.
- **Journal des révisions en ajout seul (Append-Only Log) :** Chaque delta textuel accepté est persisté comme un enregistrement immuable indexé par un numéro de séquence strictement croissant, assurant la traçabilité intégrale de l'historique et la non-régression du document.

Justification de ce choix : Le modèle NoSQL clé-valeur de DynamoDB élimine la contention de verrous sur table (*lock contention*) propre aux bases relationnelles traditionnelles lors de rafales massives, garantissant une réponse en temps constant inférieur à 10 ms pour les refus de quota.

8.2.3 Amazon SQS : découplage asynchrone de la compilation documentaire

Configuration : Une file de messages standard nommée `pdf-generation-queue`, configurée avec un délai de visibilité (*Visibility Timeout*) adapté à la durée de rendu du document (60 secondes).

Rôle au sein de l'architecture :

- **Lissage de charge (Load Leveling) :** Lors du pic de 15 000 visiteurs, la conversion d'un texte riche en document PDF constitue une opération coûteuse en ressources CPU et mémoire (temps moyen de 500 ms à 1,5 s par génération). Tenter d'exécuter ces rendus de manière synchrone lors de l'arrivée massive des requêtes provoquerait un épuisement immédiat des ressources de la machine hôte.
- **Mise en tampon des travaux admis :** Seules les requêtes ayant obtenu avec succès une place réservée lors du décrément atomique et dont la version cible n'a pas encore été compilée publient un message contenant l'identifiant du document et le numéro de séquence figé.
- **Consommation maîtrisée :** Un processus de travail d'arrière-plan (*Worker*) dépile les messages au rythme optimal de la machine, compile le document, dépose le fichier résultant sur S3, puis supprime le message de la file.

Justification de ce choix : SQS isole hermétiquement la logique d'admission rapide de la logique de calcul lourd, garantissant que l'application ne s'effondre jamais, même face à un pic d'accès cent fois supérieur à la capacité instantanée du moteur de rendu PDF.

8.2.4 Amazon S3 : stockage objet hautement disponible et URLs présignées

Configuration : Un bucket privé doté du chiffrement au repos par défaut (SSE-S3), avec blocage strict de tout accès public direct (*Block Public Access* activé).

Rôle au sein de l'architecture :

- **Stockage durable des fichiers distribués :** Conservation des fichiers PDF compilés, organisés par clé déterministe combinant l'identifiant du document et son numéro de version :

  `s3://revue-documents-storage/pdfs/doc_<id>/v_<version_seq>.pdf` (8.2)

- **Conservation des instantanés (Snapshots) :** Sauvegarde des états consolidés du document pour accélérer les opérations de reprise et de chargement initial.
- **Délégation totale de la bande passante via URLs présignées :** C'est la clé de voûte de la résistance au pic. Lorsqu'un visiteur est autorisé à télécharger, l'application ne lit pas les mégaoctets du fichier pour les retransmettre à travers sa propre connexion réseau. Elle utilise les identifiants IAM locaux pour signer cryptographiquement une URL temporaire d'accès direct à S3, valable quelques minutes.

Justification de ce choix : La génération d'une URL présignée est une opération purement mathématique calculée en mémoire locale sans aucun appel réseau. Le navigateur du visiteur télécharge ensuite le fichier directement depuis l'infrastructure d'Amazon S3. Ainsi, la bande passante réseau et la mémoire du serveur applicatif demeurent totalement préservées.

8.2.5 Amazon SNS : notification événementielle découplée

Configuration : Un sujet standard (*SNS Topic*) dédié aux alertes de distribution.

Rôle au sein de l'architecture : Dès que la transaction DynamoDB constate que la place attribuée fait passer le compteur à zéro (`places_restantes = 0`), l'application publie un message sur le sujet SNS. Ce dernier déclenche l'envoi immédiat d'un courriel électronique à l'adresse du créateur du lien.

Justification de ce choix : L'expédition de courriels n'est pas bloquante pour le traitement de la requête du visiteur. L'intégration de SNS offre une fiabilité native sans dépendre d'un serveur SMTP local complexe à administrer et soumis aux filtres anti-spam.

8.2.6 AWS IAM : sécurité par le moindre privilège et IMDSv2

Configuration : Rôle d'instance EC2 attaché à la machine virtuelle sans aucune clé d'accès statique (`aws_access_key_id` / `aws_secret_access_key`) stockée sur le disque ou dans le code source.

Rôle au sein de l'architecture : L'application interagit avec les services AWS via le service de métadonnées d'instance version 2 (IMDSv2), qui délivre des jetons d'authentification temporaires renouvelés automatiquement.

Justification de ce choix : Ce mécanisme respecte rigoureusement la règle d'évaluation interdisant toute fuite d'identifiants dans les dépôts Git et limite les droits du serveur aux seules actions strictement nécessaires (accès restreint au bucket du projet, à la table DynamoDB, à la file SQS et au sujet SNS).

8.2.7 Amazon CloudWatch : observabilité et métriques d'évaluation

Rôle au sein de l'architecture : Collecte continue des journaux système Nginx, des traces d'exécution applicatives, de l'utilisation CPU de l'instance et de la profondeur de la file SQS.

Justification de ce choix : Permet de prouver formellement aux évaluateurs le bon fonctionnement du système sous charge, la conformité des temps de réponse et l'adéquation budgétaire du projet.

8.3 Synthèse des avantages majeurs de l'architecture

Table 8.1: Synthèse des bénéfices architecturaux

Dimension	Bénéfice apporté
Atomicité absolue (C1)	Résolution de la concurrence au niveau de DynamoDB par écriture conditionnelle native ; impossibilité mathématique de surréservation.
Performance collaborative (C2)	Maintien des sockets en mémoire vive sur EC2 sans surcoût d'invocation à chaque frappe de clavier ; synchronisation CRDT en temps réel.
Immunité réseau au pic (C5)	Les flux de téléchargement des PDF sont totalement déportés vers S3 via les URLs présignées, préservant la bande passante de l'instance.
Protection contre le déni de service	La file SQS lisse la charge de compilation des PDF, protégeant l'instance de tout écroulement mémoire lors d'arrivées massives.
Sécurité sans faille	Conformité totale avec le modèle IAM IMDSv2 : aucun secret n'est codé en dur ou stocké sur le système de fichiers.
Maîtrise budgétaire	Ensemble de l'infrastructure couvert par le plan gratuit d'AWS et l'enveloppe de crédits disponible (coût facturé de 0,00 $).

Chapitre 9

Architecture Logicielle et Choix Technologiques

9.1 Vue d'ensemble du découpage applicatif

L'architecture logicielle du système est conçue pour concilier la haute réactivité exigée par l'édition collaborative en temps réel et l'arbitrage ultra-rapide d'un volume important de requêtes lors de la parution de la revue. Elle s'articule autour de trois couches logiques étanches :

1. **La couche cliente (Frontend SPA) :** une interface web monopage assurant la manipulation fluide du texte, la visualisation en direct des collaborateurs et la consultation réactive des états de téléchargement.
2. **La couche de médiation et de services applicatifs (Backend EC2) :** un ensemble de modules spécialisés hébergés sur l'instance, assurant la terminaison des protocoles réseau, la synchronisation sans perte, l'arbitrage atomique des accès et l'exécution asynchrone des compilations documentaires.
3. **La couche de persistance et de distribution (Données & Stockage) :** la combinaison d'une base de données NoSQL clé-valeur pour les données transactionnelles rapides et d'un stockage objet pour les artéfacts volumineux.

9.2 Justification détaillée des choix technologiques Frontend

Le choix de la pile frontend répond à une exigence stricte de légèreté, de rapidité d'exécution et d'intégration native avec les protocoles de synchronisation sans conflit.

9.2.1 Framework d'interface : React 18 couplé à Vite

- **Rôle :** Structuration de l'application monopage (SPA) et gestion du cycle de vie des vues (interface rédacteur, tableau de bord administrateur, écran visiteur).
- **Justification technique :**
  - *Réconciliation optimisée du DOM virtuel :* Lors d'une session collaborative à plusieurs dizaines de participants, les deltas textuels et les déplacements de curseurs génèrent des dizaines de rafraîchissements par seconde. Le mécanisme de réconciliation par lots (*automatic batching*) de React 18 évite les recalculs superflus de l'arbre DOM et préserve la fluidité de frappe sous les 60 images par seconde.
  - *Vélocité de développement avec Vite :* Vite exploite les modules ES natifs du navigateur (*ES Modules*), éliminant les temps de compilation lourds lors du développement et produisant un bundle statique hautement optimisé (minifié et pré-compressé), idéal pour être servi par Nginx avec une empreinte réseau minimale.

9.2.2 Moteur d'édition de texte riche : Quill.js (via y-quill)

- **Rôle :** Fournir aux rédacteurs une surface de saisie ergonomique (mise en forme basique, sélection, curseurs multiples colorés).

- **Justification technique :**
  - *Format Delta immuable :* Contrairement aux éditeurs basés sur du HTML brut ou l'API native instable `contenteditable`, Quill représente chaque modification sous forme de deltas linéaires (`insert`, `delete`, `retain`). Ce formalisme correspond exactement à la structure mathématique requise par les moteurs de consensus collaboratif.
  - *Liaison native avec Yjs (`y-quill`) :* La communauté open-source maintient un adaptateur officiel reliant directement l'arbre interne de Quill à la structure partagée `Y.Text`, garantissant une propagation bidirectionnelle instantanée sans écriture de code d'interfaçage complexe.

9.2.3 Moteur de synchronisation collaborative : Yjs (CRDT)

- **Rôle :** Garantir la convergence mathématique absolue de tous les rédacteurs connectés vers le même état documentaire (Contrainte C2).
- **Justification technique :**
  - *Propriétés des CRDTs (Conflict-free Replicated Data Types) :* Contrairement à l'Operational Transformation (OT) qui exige un serveur central verrouillant et réordonnant séquentiellement les frappes, Yjs applique des opérations commutatives et associatives. Même en cas de latence réseau ou de réception des paquets dans le désordre, tous les clients convergent mathématiquement vers un document strictement identique sans aucune perte de caractère.
  - *Format binaire ultra-compact :* Yjs compresse ses mises à jour sous forme d'octets bruts (`Uint8Array`) avec codage par plages (*Run-Length Encoding*), réduisant la taille moyenne d'une frappe à moins de 20 octets sur le réseau.

9.2.4 Gestion de la présence en temps réel : Sous-protocole Awareness de Yjs

- **Rôle :** Diffuser en continu les coordonnées du curseur de chaque rédacteur, son nom déclaré et une couleur d'identification unique.
- **Justification technique :**
  - *Éphémérité totale en mémoire vive :* L'information de présence n'a aucune valeur historique : elle ne doit jamais être enregistrée en base de données. Le module `Awareness` gère cette diffusion sous forme de messages volatils point-à-multipoint via les WebSockets, déchargeant intégralement la couche de persistance.
  - *Détection automatique de déconnexion :* Intègre un mécanisme de pulsation (*heartbeat*) qui supprime automatiquement l'avatar et le curseur d'un collaborateur dont la connexion est interrompue.

9.2.5 Contrôleur de téléchargement visiteur : Machine à états finis

- **Rôle :** Orchestrer l'expérience utilisateur des visiteurs accédant au lien de téléchargement lors du pic.
- **Justification technique :**
  - *Prévention des doubles soumissions :* La machine à états empêche physiquement l'utilisateur de cliquer plusieurs fois sur le bouton pendant le traitement de sa demande.
  - *Transitions déterministes :* L'interface bascule entre 4 états nets : *Vérification* (émission de la requête), *En attente de compilation* (interrogation périodique douce toutes les 1,5 s), *Prêt* (redirection immédiate vers l'URL présignée S3), et *Épuisé* (arrêt définitif de tout appel réseau).

9.3 Justification détaillée des choix technologiques Backend

9.3.1 Environnement d'exécution : Node.js LTS (Runtime V8 asynchrone)

- **Rôle :** Exécution de l'ensemble des modules applicatifs côté serveur (sockets, API REST, worker).
- **Justification technique :**
  - *Modèle d'entrées/sorties non bloquant basé sur libuv :* Un serveur synchrone multithreadé (de type Apache ou WSGI classique) alloue un thread par connexion. Pour 300 rédacteurs maintenant des sockets ouverts en permanence, la consommation mémoire et le coût de commutation de contexte (*context switching*) dégraderaient rapidement une instance `t3.micro`. Grâce à sa boucle d'événements unique (*Event Loop*), Node.js gère des milliers de connexions concurrentes avec une consommation mémoire inférieure à 150 Mo.
  - *Support natif de l'écosystème Yjs :* Le serveur officiel `y-websocket` est écrit en Node.js, garantissant une compatibilité à 100 % sans dépendance logicielle tierce.

9.3.2 Moteur de sockets collaboratif : y-websocket

- **Rôle :** Maintien des sessions WebSockets bidirectionnelles, réception des deltas CRDT et diffusion aux collaborateurs connectés sur le document.
- **Justification technique :**
  - *Synchronisation d'état en mémoire vive :* Maintient l'instance `Y.Doc` active en RAM. La fusion des modifications s'effectue en temps machine (microsecondes), permettant une diffusion inter-clients quasi-instantanée (< 2 ms).
  - *Intercepteur de contrôle de quota (C4) :* Permet d'intercepter chaque paquet entrant pour calculer le delta de caractères insérés et vérifier le crédit restant de l'auteur avant d'accepter la diffusion.

9.3.3 Passerelle HTTP & Gatekeeper : Express.js / Fastify

- **Rôle :** Exposition des routes REST d'administration, de gestion documentaire et hébergement du contrôleur d'admission des téléchargements.
- **Justification technique :**
  - *Légèreté et rapidité de mise en œuvre :* Framework minimaliste, hautement éprouvé, avec une surcharge par requête quasi-nulle.
  - *Chemin critique d'admission (Gatekeeper) :* Sous le pic de 15 000 requêtes, ce contrôleur se contente d'exécuter l'appel atomique vers DynamoDB. En cas de refus, il renvoie un code HTTP 410 en moins de 15 ms sans solliciter le système de fichiers ni le moteur de rendu documentaire.

9.3.4 Kit de développement Cloud : AWS SDK v3 pour JavaScript (@aws-sdk)

- **Rôle :** Communication entre les services applicatifs locaux et les services régionaux AWS (DynamoDB, SQS, S3, SNS).
- **Justification technique :**
  - *Architecture modulaire par client :* Contrairement à l'ancienne version monolithique du SDK, la v3 permet d'importer uniquement les clients nécessaires (`@aws-sdk/client-dynamodb`, `@aws-sdk/client-sqs`, etc.), réduisant l'empreinte mémoire du processus.

- *Résolution automatique des identifiants IMDSv2 :* Récupère de façon transparente et sécurisée les jetons temporaires fournis par le rôle d'instance IAM, sans aucune configuration de clés statiques.

9.3.5 Moteur de composition documentaire : Typst (ou Puppeteer-core Headless)

- **Rôle :** Compilation du texte consolidé de la revue en fichier PDF vectoriel haute définition lors de la première demande sur une version.
- **Justification technique :**
  - *Rapidité foudroyante et sobriété mémoire (Typst) :* Écrit en Rust, Typst compile des documents complexes en quelques dizaines de millisecondes avec une empreinte RAM inférieure à 40 Mo. Il surpasse largement les moteurs LaTeX traditionnels ou les navigateurs complets (*Headless Chrome*) qui requièrent plusieurs centaines de mégaoctets de mémoire et risqueraient de provoquer une erreur d'épuisement mémoire (*Out-Of-Memory*) sur une petite instance EC2.
  - *Typographie et rendu professionnel :* Génère un PDF conforme aux standards d'impression de la revue numérique associative.

9.3.6 Gestionnaire de processus système : systemd sous Linux

- **Rôle :** Supervision, exécution en arrière-plan et redémarrage automatique des processus Node.js.
- **Justification technique :**
  - *Résilience et haute disponibilité locale :* En configurant la directive `Restart=always` et un délai `RestartSec=3`, systemd garantit que si l'un des processus applicatifs devait s'interrompre inopinément, il est relancé en quelques secondes sans interruption prolongée du service.
  - *Injection propre de l'environnement :* Injection des variables de configuration système via un fichier dédié (`/etc/journal.env`), conforme aux exigences de sécurité du cours.

9.4 Justification détaillée des choix technologiques de Persistance

9.4.1 Base de données transactionnelle : Amazon DynamoDB (NoSQL clé-valeur)

- **Rôle :** Persistance des métadonnées, arbitrage atomique des quotas de liens (C1), journalisation en ajout seul des frappes (C2) et contrôle centralisé des crédits d'écriture (C4).
- **Justification face aux bases relationnelles (SQL) :**
  1. *Élimination de la contention de verrous (Lock Contention) :* Dans un SGBD relationnel (PostgreSQL ou MySQL), arbitrer l'attribution de 500 places parmi 15 000 requêtes en rafale impose des verrous de ligne exclusifs (`SELECT ... FOR UPDATE`). Cette approche crée une saturation immédiate de la file d'attente de verrous, une explosion des temps de latence et des risques réels d'interblocage (*deadlocks*). DynamoDB résout ce problème au niveau de son moteur Paxos distribué : l'opération conditionnelle s'exécute en temps constant $O(1)$ en moins de 10 ms sans verrouiller la table.

2. *Modélisation Single-Table Design :* Permet de regrouper l'ensemble des entités du projet au sein d'une seule table physique hautement partitionnée, réduisant drastiquement les coûts opérationnels.
3. *Facturation strictement à l'usage :* Le mode On-Demand élimine les frais d'instance permanente d'un service managé tel que RDS, garantissant une facture théorique inférieure à quelques centimes pour l'ensemble des tests.

9.4.2 Stockage des artéfacts binaires : Amazon S3

- **Rôle :** Stockage durable des fichiers PDF compilés et des instantanés consolidés des documents.
- **Justification technique :**
  - *Durabilité maximale (99,999999999 %) :* Garantit l'intégrité absolue des documents publiés.
  - *Délégation totale de bande passante par URLs présignées :* L'instance EC2 signe cryptographiquement une URL temporaire d'accès direct en mémoire locale. Le flux binaire lourd du PDF (pouvant atteindre plusieurs mégaoctets par exemplaire) est transféré directement des serveurs d'Amazon S3 vers les navigateurs des visiteurs. Ce délestage total protège la carte réseau et le processeur de l'EC2, qui restent à 100 % disponibles pour l'édition collaborative des rédacteurs.

9.4.3 File de messages tampon : Amazon SQS

- **Rôle :** File d'attente tampon asynchrone découplant la décision d'admission du visiteur de la compilation physique du fichier PDF.
- **Justification technique :**
  - *Lissage de charge (Load Leveling) :* Même si 500 visiteurs obtiennent une place dans la même seconde, la file SQS retient les ordres de rendu sans déborder. Le worker traite ces ordres à un débit constant compatible avec le CPU de la machine, éliminant tout risque d'écroulement.
  - *Gestion native des pannes :* Si la génération d'un PDF échoue, le message redevient visible dans la file après l'expiration du délai de visibilité pour être retenté, assurant une tolérance aux pannes robuste.

9.5 Serveur Frontal et Reverse-Proxy : Nginx

- **Rôle :** Point d'entrée unique de la machine virtuelle sur les ports 80 et 443.
- **Justification technique :**
  - *Terminaison TLS et sécurité :* Chiffre l'ensemble des communications (HTTPS et WSS) et protège les processus Node.js internes qui n'écoutent que sur l'interface locale `127.0.0.1`.
  - *Routage protocolaire haute performance :* Nginx gère nativement le surclassement de protocole vers les WebSockets (`proxy_set_header Upgrade $http_upgrade`) pour le chemin collaboratif, tout en relayant les requêtes API REST vers le port dédié.
  - *Mise en cache et compression :* Compresse les ressources web statiques (Gzip) pour accélérer le chargement initial de l'application cliente sur les navigateurs des rédacteurs.

9.6 Matrice de synthèse des choix technologiques

Table 9.1: Synthèse des composants logiciels et justifications associées

Composant	Technologie	Justification clé
Interface Rédacteur	React 18 + Vite	Réactivité de l'arbre virtuel sous frappes intenses ; outillage moderne et rapide.
Éditeur de texte	Quill.js + y-quill	Modèle Delta immuable parfaitement adapté aux flux de synchronisation temps réel.
Consensus d'édition	Yjs (CRDT)	Convergence déterministe mathématique sans verrou ni perte de caractère (C2).
Présence	Awareness Protocol	Diffusion éphémère ultra-légère des curseurs en mémoire vive sans écriture disque.
Serveur Applicatif	Node.js (LTS)	Modèle événementiel asynchrone idéal pour 300 sockets persistants sur petite RAM.
Serveur Collab	y-websocket	Serveur officiel Yjs optimisé pour le protocole binaire compact.
Passerelle REST	Express / Fastify	Simplicité, robustesse et temps de réponse $<$ 15 ms sur le Gatekeeper d'admission.
Compilateur PDF	Typst / Puppeteer	Vitesse de compilation native en millisecondes et très faible consommation mémoire.
Supervision	systemd Linux	Haute disponibilité locale et reprise automatique sur incident sans intervention.
Base transactionnelle	Amazon DynamoDB	Écritures conditionnelles atomiques $O(1)$ sans contention de verrous (C1 & C4).
Stockage des fichiers	Amazon S3	URLs présignées déportant 100 % de la bande passante du téléchargement (C5).
Tampon asynchrone	Amazon SQS	Découplage strict et lissage de charge évitant tout écroulement CPU de l'instance.
Reverse-Proxy	Nginx	Terminaison TLS, compression et routage sélectif WSS / HTTP sur ports standards.

Conclusion

Ce rapport a présenté la conception intégrale d'un éditeur de documents partagé pour une association produisant une revue numérique collaborative. Il a identifié les trois acteurs du système, délimité le périmètre fonctionnel, formalisé les exigences fonctionnelles et non fonctionnelles (avec un focus sur les contraintes critiques C1 à C6), énoncé les principes métier structurants, et fourni une analyse de conception détaillée.

L'architecture cloud AWS retenue concilie de manière pragmatique et performante les deux régimes d'accès du système :

- Une voie d'édition collaborative temps réel basée sur une instance EC2 légère et le protocole de sockets *y*-websocket, offrant une latence minimale et une convergence mathématique sans verrou grâce aux CRDTs (C2) ;
- Une voie de distribution documentaire à haute disponibilité reposant sur l'atomicité native d'Amazon DynamoDB pour garantir le respect strict du quota sans surréservation (C1), le découplage asynchrone des compilations lourdes via Amazon SQS, et le délestage intégral de la bande passante grâce aux URLs présignées Amazon S3.

Cette architecture tire parti des bonnes pratiques acquises en cours, respecte les principes de sécurité du moindre privilège via les rôles IAM IMDSv2, et s'exécute sans aucun surcoût dans le cadre des crédits alloués. Les tests de charge et de convergence associés permettront de prouver formellement la tenue de l'ensemble des critères d'évaluation sous les conditions réelles de trafic.

— Fin du rapport —